



Planejador Financeiro

Manual de Implantação — Vercel & Supabase

 Guia com foco em segurança

Documento técnico de implantação

Versão 1.0 • Agosto de 2026

React 18 • Vite 5 • Supabase • Vercel

Público: responsável pela implantação

Pré-requisito técnico: nenhum

Sumário

1. Antes de começar

O que você vai precisar • Tempo estimado • Regra de ouro das credenciais

2. Como a segurança deste sistema funciona

Por que a chave pública não é um vazamento • As camadas de proteção

3. Parte 1 — Configurar o Supabase (banco de dados)

Criar conta e projeto • Executar o schema • Verificar a proteção • Configurar autenticação

4. Parte 2 — Obter as credenciais

Onde encontrar • Quais copiar • Qual NUNCA usar

5. Parte 3 — Publicar na Vercel

Enviar o código • Importar o projeto • Cadastrar as variáveis • Publicar

6. Parte 4 — Ajustes finais de segurança

URLs de redirecionamento • Proteção contra abuso • Confirmação de e-mail

7. Parte 5 — Validação pós-implantação

Roteiro de testes • Teste de isolamento entre contas

8. Rodar o projeto no seu computador (opcional)

9. Monitoramento e manutenção

Painel de segurança • O que observar • Rotina recomendada

10. Solução de problemas

Erros mais comuns e como resolver

11. Referência rápida — onde inserir cada credencial

12. Checklist final de implantação

1. Antes de começar

Este manual leva você do zero até a aplicação publicada na internet. Cada passo indica exatamente onde clicar e o que preencher. Não é necessário saber programar.

O que você vai precisar

Item	Observação
Conta no Supabase	Gratuita — supabase.com
Conta na Vercel	Gratuita — vercel.com
Conta no GitHub	Gratuita — usada para enviar o código à Vercel
Aplicativo autenticador	Opcional, recomendado. Google Authenticator, Authy ou 1Password

Tempo estimado

Entre **30 e 45 minutos** na primeira vez, incluindo a criação das contas.

⚠ Regra de ouro das credenciais

Nenhuma chave, token ou arquivo `.env` deve ser enviado por e-mail, chat, mensagem ou compartilhado com terceiros — inclusive com quem prestou suporte técnico ao projeto.

Este manual indica

o local exato

onde cada credencial deve ser inserida. Você faz isso sozinho, direto nos painéis do Supabase e da Vercel. Nenhum passo aqui pede que você entregue uma chave a alguém.

2. Como a segurança deste sistema funciona

Entender este ponto evita um susto desnecessário mais adiante.

A chave pública aparece no navegador — e isso está correto

A aplicação usa duas informações para falar com o banco: a **URL do projeto** e a **chave anônima** (*anon key*). Ambas ficam visíveis no código que roda no navegador. Qualquer visitante consegue lê-las.

Isso **não é uma falha**. É o funcionamento documentado e esperado do Supabase para aplicações web. A chave anônima sozinha não dá acesso a nada: ela apenas identifica o projeto.

💡 Analogia

A chave anônima é como o **endereço de um prédio**

. Saber o endereço não abre nenhum apartamento. O que protege cada apartamento é a fechadura — e no nosso caso a fechadura é o Row Level Security, configurado no banco de dados.

As camadas de proteção implementadas

Camada	O que faz
Row Level Security (RLS)	O banco recusa qualquer leitura ou escrita de dados que não pertençam ao usuário logado. É a proteção principal.
Trigger anti-sequestro	Impede que um registro seja transferido para outro usuário, mesmo por comando direto no banco.
Registro de auditoria	Toda tentativa de acesso indevido é gravada em tabela imutável — nem o próprio usuário consegue apagar.
Verificação em duas etapas	MFA por aplicativo autenticador, opcional e ativável pelo usuário.
Bloqueio por tentativas	Cinco senhas erradas bloqueiam novas tentativas por 5 minutos.
Encerramento por inatividade	A sessão cai sozinha após 30 minutos sem uso.
Mensagens neutras	O sistema nunca revela se um e-mail existe na base, impedindo mapeamento de usuários.
Cabeçalhos HTTP	CSP, HSTS e anti-clickjacking configurados no arquivo <code>vercel.json</code> .
Validação em duas pontas	Dados conferidos na tela e novamente pelo banco — a palavra final é sempre do banco.

🚫 A chave que nunca deve ser usada

O Supabase oferece uma segunda chave chamada `service_role`. Ela **ignora todas as regras de segurança** e daria acesso irrestrito aos dados de todos os usuários.

Esta aplicação

não usa e nunca deve usar

essa chave. Se em algum momento você for orientado a colá-la em uma variável começando com `VITE_`, **recuse**: isso a exporia publicamente no navegador.

3. Parte 1 — Configurar o Supabase

1 Criar a conta e o projeto

1. Acesse <https://supabase.com> e clique em **Start your project**.
2. Entre com sua conta GitHub (ou crie uma conta com e-mail).
3. No painel, clique em **New project**.
4. Preencha os campos:

Name

`planejador-financeiro` — ou outro nome de sua preferência.

Database Password

Clique em **Generate a password** e **guarde essa senha em um gerenciador de senhas**.

Esta é a senha do banco. A aplicação não a utiliza, mas você precisará dela para manutenção avançada. Ela não pode ser recuperada depois — apenas redefinida.

Region

Escolha a mais próxima dos usuários. Para o Brasil: **South America (São Paulo)**.

A região afeta a velocidade de resposta e não pode ser alterada depois.

5. Clique em **Create new project** e aguarde de 2 a 3 minutos.

2 Executar o script do banco de dados

Este é o passo mais importante do manual. Ele cria as tabelas e — principalmente — **ativa toda a proteção de dados**.

1. No menu lateral do Supabase, clique em **SQL Editor**.
2. Clique em **New query**.
3. Abra o arquivo `supabase/schema.sql` da pasta do projeto em um editor de texto (Bloco de Notas serve).
4. Selecione **todo** o conteúdo (`Ctrl+A`), copie (`Ctrl+C`) e cole no SQL Editor (`Ctrl+V`).
5. Clique em **Run** (ou pressione `Ctrl+Enter`).

✓ Como saber que deu certo

Ao final da execução aparece uma tabela de resultados com seis linhas — `budgets` , `categories` , `goals` , `profiles` , `security_events` e `transactions` — e a coluna **rls_ativa** deve mostrar `true` em todas elas.

● Se alguma linha mostrar `false`

Pare a implantação.

Uma tabela sem RLS é uma tabela desprotegida. Execute o arquivo novamente por completo — ele foi escrito para poder ser reexecutado sem causar dano. Se ainda assim algum `false` persistir, não prossiga para a publicação.

3 Configurar a autenticação

Vá em **Authentication** → **Providers** no menu lateral.

1. Confirme que **Email** está habilitado.
2. Desabilite todos os outros provedores que não forem utilizados (Google, GitHub, etc.). Cada provedor ativo é uma porta a mais.

Confirmação de e-mail

Vá em **Authentication** → **Sign In / Providers** → **Email** e localize a opção **Confirm email**.

⚠️ Decisão sua — leia antes de escolher

Desligado

(conforme solicitado): o usuário cria a conta e entra imediatamente. Mais prático, porém qualquer pessoa pode se cadastrar com um e-mail que não lhe pertence.

Ligado

(recomendado para uso real): o usuário precisa clicar em um link enviado por e-mail antes de acessar. Impede cadastro com e-mail de terceiros e garante que a recuperação de senha funcione.

A aplicação já funciona nos dois modos, sem alteração de código. Você pode começar desligado e ativar depois — a mudança vale para os cadastros seguintes.

4. Parte 2 — Obter as credenciais

Agora você vai copiar os dois valores que a aplicação usa para se conectar ao banco.

1. No Supabase, clique na engrenagem **Project Settings** (menu lateral, parte inferior).
2. Clique em **API**.

Os dois valores que você vai usar

Campo no painel	Aparência	Onde será usado
Project URL	<code>https://abcd...xyz.supabase.co</code>	Variável <code>VITE_SUPABASE_URL</code>
anon / public	Texto longo iniciado por <code>eyJ...</code>	Variável <code>VITE_SUPABASE_ANON_KEY</code>

Copie os dois para um bloco de notas temporário. Você os usará no próximo capítulo.

⚠ Na mesma tela existe uma chave perigosa

Logo abaixo você verá

`service_role`

, marcada como *secret*. **Não copie essa chave.** Ela ignora todas as proteções do banco.

Se copiá-la por engano, apague imediatamente do bloco de notas. Se ela chegar a ser publicada em qualquer lugar, use o botão

Reset

no painel do Supabase para invalidá-la.

💡 Como distinguir as duas chaves

Ambas começam com `eyJ`. A diferença está no rótulo ao lado: `anon` `public` é a correta; `service_role` `secret` é a que nunca deve sair do painel.

5. Parte 3 — Publicar na Vercel

1 Enviar o código para o GitHub

A Vercel publica a aplicação a partir de um repositório. Se o projeto ainda não estiver no GitHub:

1. Acesse `https://github.com/new`.
2. Dê um nome ao repositório, por exemplo `planejador-financeiro`.
3. Marque a opção **Private**.
4. Clique em **Create repository** e siga as instruções exibidas para enviar os arquivos.

✓ Proteção já configurada

O arquivo `.gitignore` do projeto já impede que `.env`, chaves e certificados sejam enviados ao GitHub por acidente. Ainda assim, antes de publicar, confirme que **nenhum arquivo chamado `.env` aparece** na lista de arquivos do repositório.

2 Importar o projeto na Vercel

1. Acesse <https://vercel.com> e entre com sua conta GitHub.
2. Clique em **Add New...** → **Project**.
3. Localize o repositório na lista e clique em **Import**.

A Vercel detecta o Vite automaticamente. Confirme que os campos estão assim:

Campo	Valor
Framework Preset	Vite
Build Command	npm run build
Output Directory	dist
Install Command	npm install

3 Cadastrar as variáveis de ambiente

⚠ Não pule esta etapa

Sem as variáveis, a aplicação publica normalmente mas exibe a mensagem "Conexão com o banco de dados não configurada" em vez da tela de login.

Ainda na tela de importação, expanda a seção **Environment Variables** e cadastre as duas variáveis:

Variável 1

Name: `VITE_SUPABASE_URL`

Value: a *Project URL* copiada do Supabase

Environments: marque `Production`, `Preview` e `Development`

Variável 2

Name: `VITE_SUPABASE_ANON_KEY`

Value: a chave *anon* / *public* copiada do Supabase

Environments: marque `Production`, `Preview` e `Development`

⚠️ Atenção ao prefixo `VITE_`

Toda variável iniciada por `VITE_` é **embutida no código enviado ao navegador** e fica visível a qualquer visitante. Isso é aceitável para estas duas — e apenas para elas.

Nunca

cadastre com o prefixo `VITE_` uma senha de banco, a chave `service_role`, um token de API ou qualquer outro segredo.

4 Publicar

1. Clique em **Deploy**.
2. Aguarde de 1 a 2 minutos.
3. Ao concluir, a Vercel exibe o endereço da aplicação, no formato `https://seu-projeto.vercel.app`.

Copie esse endereço — ele é necessário no próximo capítulo.

✅ Proteções aplicadas automaticamente

O arquivo `vercel.json` incluído no projeto aplica sozinho, a cada publicação: HTTPS obrigatório (HSTS), bloqueio de incorporação em outros sites (anti-clickjacking), política de conteúdo restrita (CSP) e bloqueio de acesso a câmera, microfone e localização.

6. Parte 4 — Ajustes finais de segurança

Com o endereço da Vercel em mãos, volte ao Supabase para concluir a configuração.

1 Autorizar o endereço da aplicação

Sem este passo, a recuperação de senha não funciona e o Supabase pode recusar as requisições.

No Supabase, vá em **Authentication** → **URL Configuration**:

Site URL

Cole o endereço da Vercel:

`https://seu-projeto.vercel.app`

Sem barra no final.

Redirect URLs

Adicione as duas linhas abaixo:

`https://seu-projeto.vercel.app/**`

`http://localhost:5173/**`

A segunda só é necessária se você for rodar o projeto no seu computador. Em ambiente de produção puro, remova-a.

⚠ Nunca use curinga aberto

Jamais cadastre `*` sozinho neste campo. Isso permitiria que um site malicioso recebesse o link de recuperação de senha dos seus usuários.

2 Reduzir a duração da sessão

Em **Authentication** → **Sessions**:

Configuração	Valor sugerido
Access token expiry	3600 segundos (1 hora)
Refresh token rotation	Habilitado
Reuse interval	10 segundos

A rotação de token faz com que uma credencial roubada perca a validade rapidamente. A aplicação já encerra a sessão após 30 minutos de inatividade, de forma independente.

3 Limitar tentativas automatizadas

Em **Authentication** → **Rate Limits**, confirme que os limites padrão estão ativos. Se o painel permitir ajuste, valores recomendados:

Limite	Valor sugerido
Sign in / Sign up	30 por hora, por IP
Token refresh	150 por hora, por IP
E-mails enviados	30 por hora

Defesa em duas frentes

A aplicação bloqueia o usuário após 5 senhas erradas, mas essa trava vive no navegador e um atacante determinado pode contorná-la. O limite configurado aqui roda no

servidor

do Supabase e é a barreira que realmente detém um ataque automatizado.

4 Ativar proteção contra senhas vazadas

Em **Authentication** → **Policies** (ou **Password Settings**, conforme a versão do painel):

- Ative **Leaked password protection** — impede o uso de senhas que já apareceram em vazamentos públicos conhecidos.
- Defina o comprimento mínimo como **10** caracteres, para espelhar a regra já aplicada na tela de cadastro.

7. Parte 5 — Validação pós-implantação

Execute este roteiro antes de considerar a implantação concluída.

Roteiro de testes

#	Teste	Resultado esperado
1	Abrir o endereço da Vercel	Tela de login é exibida, sem aviso de configuração pendente
2	Criar uma conta com senha fraca (123456)	O cadastro é recusado e os requisitos aparecem na tela
3	Criar uma conta com senha forte	Acesso concluído e categorias padrão já carregadas
4	Cadastrar um lançamento e recarregar a página	O lançamento continua lá
5	Errar a senha 5 vezes seguidas	Mensagem de bloqueio temporário aparece
6	Abrir a aba Segurança	Histórico lista os eventos de login
7	Ativar a verificação em duas etapas	QR code é exibido e o código do aplicativo é aceito
8	Sair e entrar novamente	O código de 6 dígitos é solicitado
9	Abrir a aplicação pelo celular	Layout se adapta; nenhum elemento cortado
10	Solicitar recuperação de senha	E-mail chega e o link abre a aplicação

Teste de isolamento entre contas

Este é o teste mais importante do manual: ele confirma na prática que um usuário não enxerga os dados de outro.

1. Crie uma segunda conta com um e-mail diferente.
2. Cadastre nela um lançamento com descrição facilmente identificável, por exemplo **TESTE CONTA B**.
3. Saia e entre novamente com a **primeira** conta.
4. Procure por **TESTE CONTA B** na lista de lançamentos.

✓ Resultado esperado

O lançamento da segunda conta

não aparece

em lugar nenhum. Cada conta enxerga exclusivamente os próprios dados.

● Se o lançamento aparecer

O Row Level Security não está ativo.

Interrompa o uso imediatamente

, volte ao capítulo 3 e execute o arquivo `schema.sql` novamente, conferindo que todas as seis linhas retornem `rls_ativa = true`.

8. Rodar o projeto no seu computador (opcional)

Necessário apenas se você quiser testar alterações antes de publicar.

1. Instale o Node.js versão 18 ou superior — <https://nodejs.org>.
2. Na pasta do projeto, localize o arquivo `.env.example`.
3. Faça uma cópia dele e renomeie a cópia para `.env` (apenas isso, sem nada antes do ponto).
4. Abra o `.env` e preencha os dois valores:

```
VITE_SUPABASE_URL=https://seu-projeto.supabase.co
VITE_SUPABASE_ANON_KEY=eyJ...sua-chave-anon-aqui
```

5. Abra o terminal na pasta do projeto e execute:

```
npm install
npm run dev
```

A aplicação abre em <http://localhost:5173>.

⚠ O arquivo `.env` nunca deve ser compartilhado

Ele já está protegido pelo `.gitignore` e não será enviado ao GitHub. Não o anexe em e-mails, não o cole em conversas e não o envie a prestadores de suporte.

9. Monitoramento e manutenção

O painel de segurança da aplicação

A aba **Segurança**, dentro da aplicação, mostra o histórico de eventos da conta. Os registros são **imutáveis**: nem o próprio usuário consegue editar ou apagar, o que impede que um invasor apague os próprios rastros.

Evento	O que significa
 Login realizado	Acesso bem-sucedido
 Tentativa de login falhou	Senha incorreta
 Código de verificação incorreto	Falha no MFA — atenção se for recorrente
 Tentativa de acesso a dados de outro usuário	Sinal grave. Alguém tentou ler dados alheios e o banco bloqueou
 Verificação em duas etapas desativada	Atenção se você não fez isso
 Exclusão em massa de dados	Atenção se você não fez isso

Quando três ou mais eventos de atenção ocorrem em 24 horas, um alerta aparece automaticamente no topo do painel.

O que fazer diante de um evento suspeito

1. Troque a senha imediatamente.
2. Ative a verificação em duas etapas, se ainda não estiver ativa.
3. No Supabase, vá em **Authentication** → **Users**, localize a conta e encerre as sessões ativas.
4. Baixe um backup dos dados pela aba **Configurações**.

Rotina recomendada

Frequência	Ação
Semanal	Conferir a aba Segurança em busca de eventos não reconhecidos
Mensal	Baixar backup JSON pela aba Configurações
Mensal	Revisar a lista de usuários em Supabase → Authentication → Users
Trimestral	Executar <code>npm update</code> e publicar novamente, para aplicar correções de segurança das bibliotecas
Semestral	Reexecutar o teste de isolamento entre contas (capítulo 7)

Backup do banco

O plano gratuito do Supabase mantém backups automáticos por 7 dias. Para retenção maior, use a exportação JSON da aplicação — ela preserva lançamentos, categorias, orçamentos e metas.

10. Solução de problemas

"Conexão com o banco de dados não configurada"

Causa: as variáveis de ambiente não foram cadastradas, estão com o nome errado, ou o site não foi republicado após o cadastro.

Solução: na Vercel, vá em **Settings** → **Environment Variables** e confirme que existem exatamente `VITE_SUPABASE_URL` e `VITE_SUPABASE_ANON_KEY` — sem espaços antes ou depois do nome. Depois vá em **Deployments**, abra o mais recente e clique em **Redeploy**.

⚠ Ponto que causa mais confusão

Na Vercel, variáveis de ambiente

só passam a valer após uma nova publicação

. Cadastrar a variável e recarregar a página não resolve — é preciso republicar.

"E-mail ou senha incorretos" mesmo com a senha certa

Causa provável: a opção *Confirm email* está ligada e o e-mail ainda não foi confirmado.

Solução: procure o e-mail de confirmação, inclusive na caixa de spam. Alternativamente, em **Supabase** → **Authentication** → **Users**, localize o usuário e confirme manualmente.

A página abre em branco

Solução: pressione `F12` para abrir o console do navegador e verifique a mensagem exibida. Na Vercel, confira em **Deployments** se a publicação terminou com *Ready* e não com *Error*.

"Acesso negado pelas regras de segurança do banco de dados"

Causa: o arquivo `schema.sql` não foi executado por completo.

Solução: repita o capítulo 3, passo 2, colando o arquivo inteiro. Confirme que a verificação final retorna `rls_ativa = true` nas seis linhas.

O e-mail de recuperação de senha não chega

Solução: confira a caixa de spam. Verifique se o **Site URL** foi configurado (capítulo 6). O serviço de e-mail gratuito do Supabase tem limite de envios por hora e é destinado a testes — para uso real, configure um SMTP próprio em **Project Settings** → **Auth** → **SMTP Settings**.

O QR code do MFA não aparece

Solução: em **Supabase** → **Authentication** → **Providers**, confirme que a opção **Multi-Factor Authentication (TOTP)** está habilitada.

Layout quebrado no celular

Solução: limpe o cache do navegador. A publicação mais recente inclui os ajustes de layout para telas pequenas, incluindo suporte a aparelhos com entalhe (notch).

11. Referência rápida — onde inserir cada credencial

Resumo consolidado de todos os pontos em que você insere alguma informação. Nenhum deles exige entregar credencial a terceiros.

Credencial	Onde inserir	Observação
Project URL	Vercel → Settings → Environment Variables → <code>VITE_SUPABASE_URL</code>	Pública por natureza
Chave anon	Vercel → Settings → Environment Variables → <code>VITE_SUPABASE_ANON_KEY</code>	Pública por natureza; protegida pelo RLS
Project URL (local)	Arquivo <code>.env</code> na raiz do projeto	Somente se rodar no seu computador
Chave anon (local)	Arquivo <code>.env</code> na raiz do projeto	Nunca versionar
Senha do banco	Apenas no seu gerenciador de senhas	A aplicação não a utiliza
Endereço da Vercel	Supabase → Authentication → URL Configuration	Site URL e Redirect URLs
service_role	Em lugar nenhum	Nunca sai do painel do Supabase

Arquivos do projeto e sua função

Arquivo	Função
<code>supabase/schema.sql</code>	Cria tabelas e ativa toda a proteção. Executar no SQL Editor
<code>vercel.json</code>	Cabeçalhos de segurança e cache. Aplicado automaticamente
<code>.env.example</code>	Modelo de configuração local. Não contém segredo
<code>.env</code>	Criado por você, apenas local. Nunca versionar
<code>.gitignore</code>	Impede envio acidental de segredos ao GitHub
<code>src/lib/supabase.js</code>	Conexão com o banco. Não editar para inserir chaves
<code>src/lib/audit.js</code>	Registro de eventos de segurança

💡 Por que não editar `src/lib/supabase.js`

Esse arquivo lê as credenciais das variáveis de ambiente. Escrever a chave diretamente nele faria com que ela fosse enviada ao GitHub no próximo commit — exatamente o que as variáveis de ambiente existem para evitar.

12. Checklist final de implantação

Supabase

- ☐ Projeto criado e senha do banco guardada em gerenciador de senhas
- ☐ Arquivo `schema.sql` executado por completo
- ☐ Verificação final retornou `rls_ativa = true` nas **seis** tabelas
- ☐ Provedor de e-mail habilitado; provedores não utilizados desabilitados
- ☐ Opção *Confirm email* definida conforme sua decisão
- ☐ Site URL preenchida com o endereço da Vercel
- ☐ Redirect URLs cadastradas sem curinga aberto
- ☐ Rotação de refresh token habilitada
- ☐ Proteção contra senhas vazadas ativada
- ☐ MFA (TOTP) habilitado nos provedores

Vercel

- ☐ Repositório privado importado
- ☐ `VITE_SUPABASE_URL` cadastrada nos três ambientes
- ☐ `VITE_SUPABASE_ANON_KEY` cadastrada nos três ambientes
- ☐ Nenhuma variável `VITE_` contém segredo real
- ☐ Publicação concluída com status *Ready*
- ☐ Arquivo `vercel.json` presente no repositório

Validação funcional

- ☐ Tela de login abre sem aviso de configuração pendente
- ☐ Senha fraca é recusada no cadastro
- ☐ Conta criada e categorias padrão carregadas
- ☐ Lançamento persiste após recarregar a página
- ☐ Bloqueio após 5 tentativas de senha funciona
- ☐ Aba Segurança exibe o histórico de eventos
- ☐ MFA ativa e solicita código no login seguinte
- ☐ **Teste de isolamento aprovado — conta A não vê dados da conta B**
- ☐ Layout íntegro no celular
- ☐ Recuperação de senha entrega o e-mail

Segurança geral

- ☐ Chave `service_role` nunca foi copiada nem colada em lugar algum
- ☐ Nenhum arquivo `.env` aparece no repositório do GitHub
- ☐ Repositório configurado como privado
- ☐ Senha do banco armazenada apenas em gerenciador de senhas
- ☐ Nenhuma credencial foi enviada por e-mail, chat ou mensagem

✓ Implantação concluída

Com todos os itens marcados, a aplicação está publicada e protegida. Guarde este manual: os capítulos 9 e 10 continuam úteis durante a operação.